

Cluster-Wide Task Slowdown Detection in Cloud System

Feiyi Chen , Yingying Zhang , Lunting Fan , Yuxuan Liang , Guansong Pang ,
Qingsong Wen , Shuiguang Deng

KDD 2024

24 August, 2024

Outline

- 1. Background
- 2. Challenges & Contributions
- 3. Preliminary & Methodology
- 4. Experiments
- 5. Conclusion

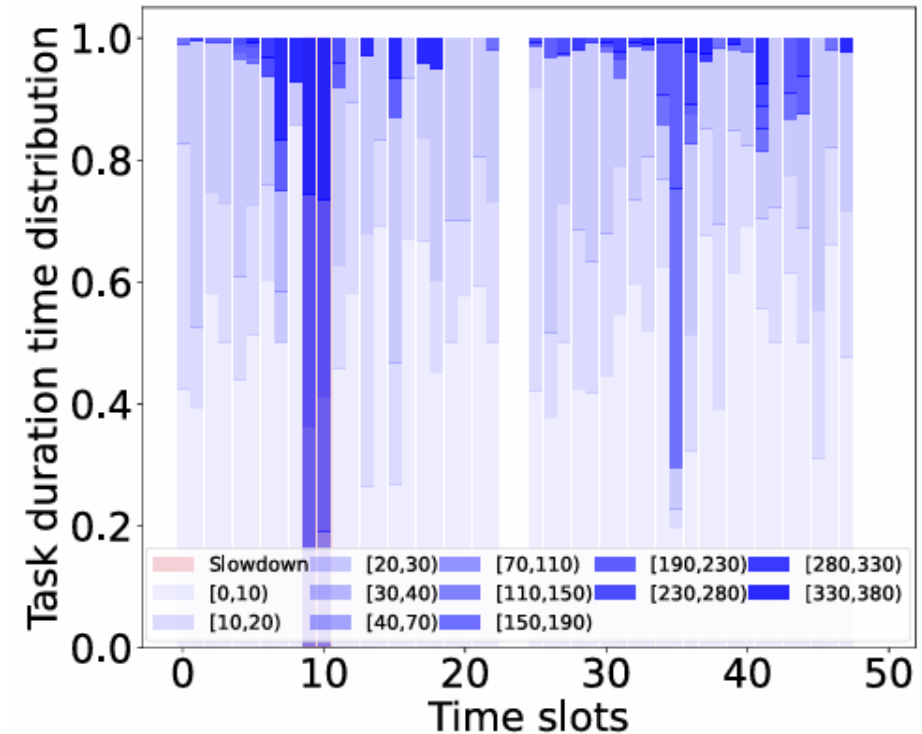
1. Research Background

Slow task detection is a critical problem in cloud operation and maintenance since it is highly related to user experience and can bring substantial liquidated damages.

- Most anomaly detection methods detect it from a single-task aspect. However, considering millions of concurrent tasks in large-scale cloud computing clusters , it becomes impractical and inefficient.
- Single-task slow-downs are very common and do not necessarily indicate a malfunction of a cluster due to its violent fluctuation nature in a virtual environment.

1. Research Background

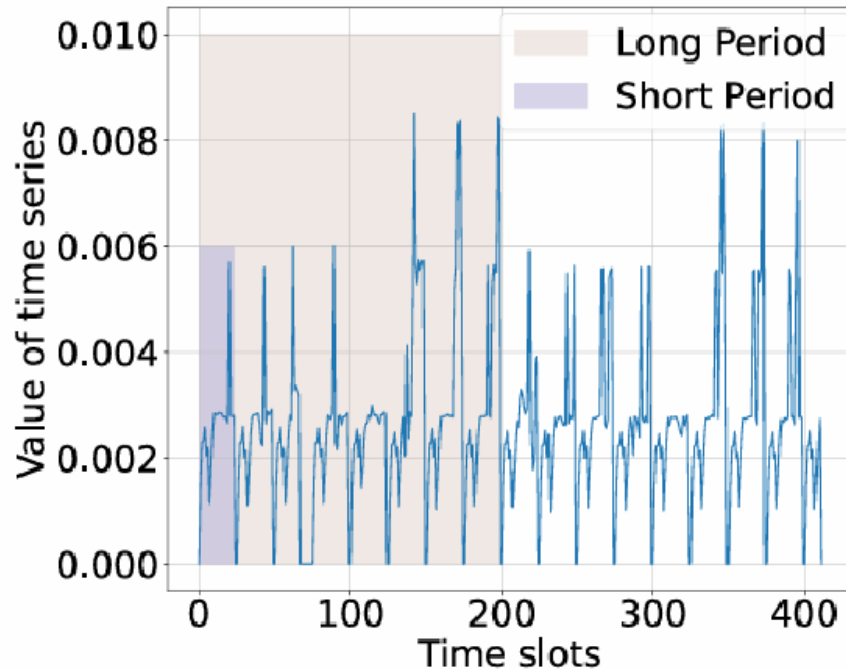
We pivot towards detecting slowdowns on a cluster-wide scale. Particularly, we detect cluster-wide task slowdowns using the duration time distribution of a cluster, as illustrated in Fig. 1(a), in which for each time slot we partition the range of task duration time into intervals and calculate the proportion of tasks falling into each interval. This strategic shift not only significantly reduces the computational complexity of our algorithm, making it independent of the number of tasks, but also enhances the accuracy of cluster malfunction detection.



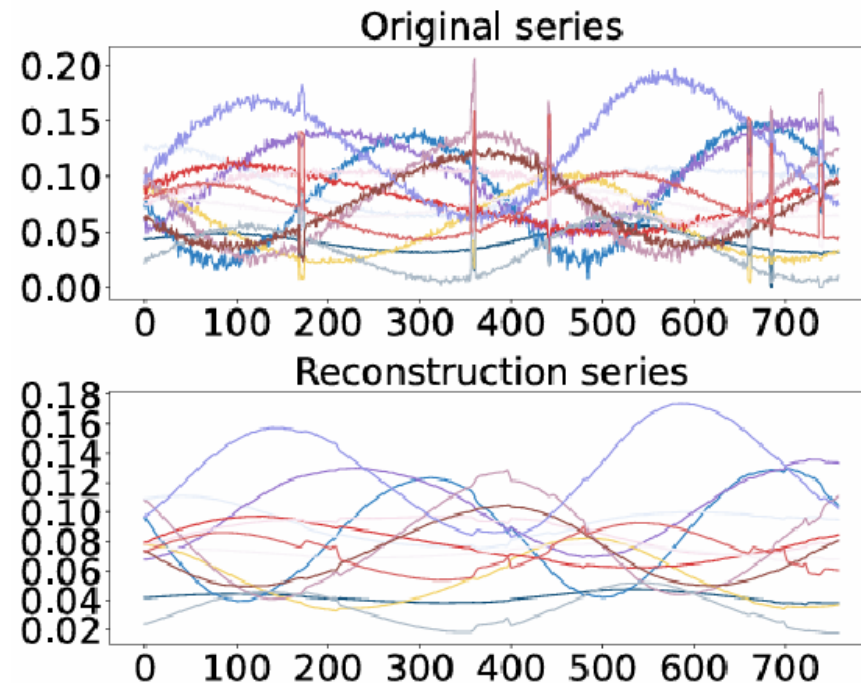
(a) The task duration time distribution

2.1 Challenges

- **Compound periodicity:** different tasks show different periodicities → the distribution of cluster-wide tasks show compound periodicity



(b) The compound periodicity



(c) The reconstruction series of attention

2.1 Challenges

- **Non-slowing exceptional fluctuation:** In many application scenario (e.g. slow sql detection etc.), we are only interested on slowdown tasks.
- **Noisy training data:** In unsupervised anomaly detection, it is assumed there is no anomalies in training data, which is impractical since the distribution contains a huge number of tasks and it is hard to assure all of them are normal.

2.2 Contributions

- We present the first attempt to formalize the cluster-wide slow down problem with the identification of the problem specifications and relevant challenges.
- We introduce a novel skimming attention mechanism.
- We introduce a novel Neural OT module tailored to reconstruct the normal non-periodic fluctuations observed in the duration time distribution, while effectively filtering out the cluster-wide slow-down anomalies.
- We propose a picky loss function that assigns higher weights to reliable time slots within the loss function
- We conducted extensive experiments and demonstrated that our method (SORN) outperforms the state-of-the-art (SOTA) methods in F1 score on real-world industrial datasets.

3.1.1 Optimal Transport (OT)

$I = \{(s_1, s_2], (s_2, s_3], \dots, (s_{n-1}, s_n]\}$ two distributions $\mathbf{a} \in R^N$ and $\mathbf{b} \in R^N$,

- where $a_i = P(s_i < x \leq s_{i+1})$, $b_i = P(s_i < x \leq s_{i+1})$.
- The Optimal Transport problem aims at transforming distribution $\mathbf{a}$ to $\mathbf{b}$ by moving a fraction of the amount in each interval of $\mathbf{a}$ to another interval.
- Moving a unit from j^{th} interval to i^{th} interval costs a price $C_{i,j}$.
- The Optimal Transport problem gropes for an optimal transport strategy P costing the lowest price, where $P_{i,j}$ denotes the amount of unit moving from j^{th} interval to the i^{th} , as shown in Eq.1, where $\langle P, C \rangle$ denotes the Frobenius dot-product.

$$\begin{aligned} & \min_P \langle P, C \rangle, \\ & s.t. \ P \cdot \vec{1} = \mathbf{b}, \ P^T \cdot \vec{1} = \mathbf{a}. \end{aligned} \tag{1}$$

3.1.2 Problem Setup

Definition 1. f_t and f_t^* are used to denote the real-time distribution and expected distribution at time slot t . $f_t(\alpha)$ and $f_t^*(\alpha)$ are used to denote the α -quantile of distribution f_t and f_t^* . $\mathcal{T}$ is used to denote the threshold for tolerable fluctuation range of duration time distribution.

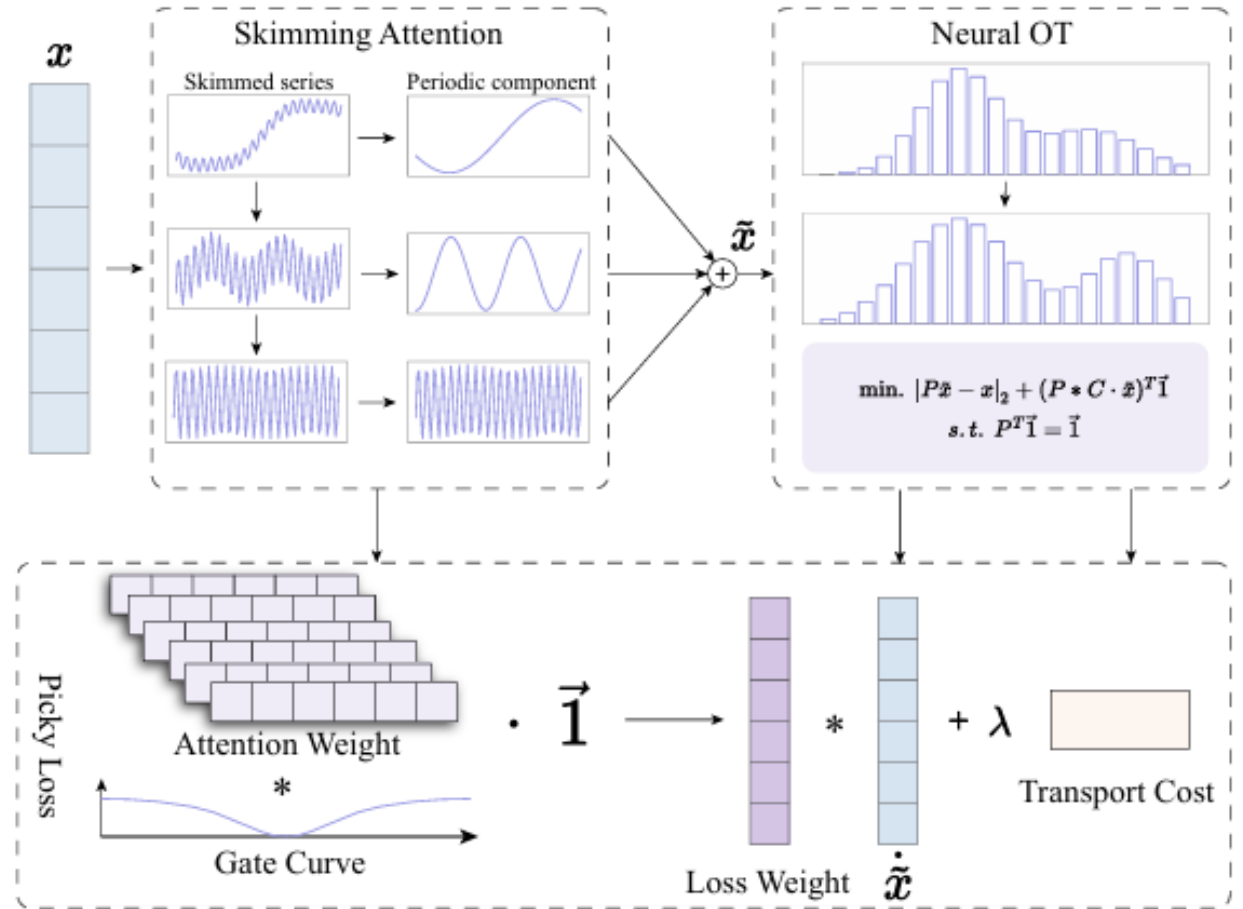
Definition 2. If there is a slowdown at time slot t , then $\max_{\alpha} f_t(\alpha) - f_t^*(\alpha) > \mathcal{T}, \forall \alpha$.

Definition 3. (Input data & output data) Given a set of intervals $I = \{[s_1, s_2), [s_2, s_3), \dots, [s_D, s_{D+1})\}$, the input data is a T -length and D dimensional multivariate time series $x \in R^{T \times D}$, where $x[t, d]$ is the number of tasks whose duration time falls into the d^{th} interval $[s_d, s_{d+1})$. The reconstruction series of SORN is denoted by $\hat{x}$.

Problem Formalization. We use a SORN to obtain a reconstruction series $\hat{x}$ from the original input data x . Subsequently, we use an anomaly score function $\text{AnomalyScore}(\hat{x}, x, I)$. We aim to maximize the anomaly score gap between the slow-down time slots and the others.

3.2 SORN Overview

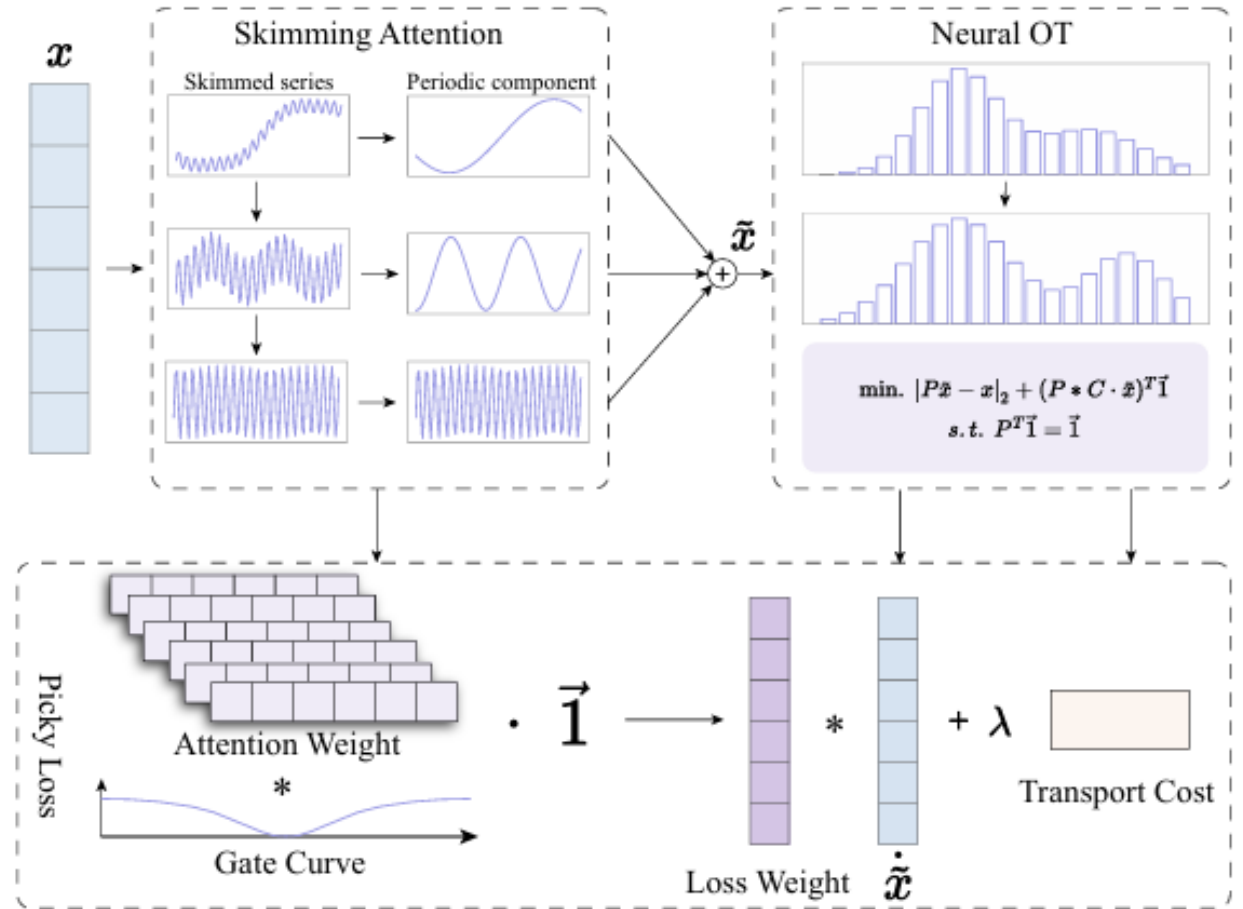
- We sequentially mask each time slot in x and employ a multi-layer Skimming Attention mechanism to reconstruct the time slot by leveraging compound periodic information.
- We utilize Neural OT to fine-tune the reconstructed series obtained from Skimming Attention, capturing aperiodic but typical fluctuations in the time series.



(a) The model architecture of SORN

3.2 SORN Overview

- We apply the picky loss function to assign higher weights to normal time slots while assigning lower weights to occasional anomalous slots in the loss function.



(a) The model architecture of SORN

3.2.1 Skimming Attention

- We first use a sliding window with padding to extend the input data.

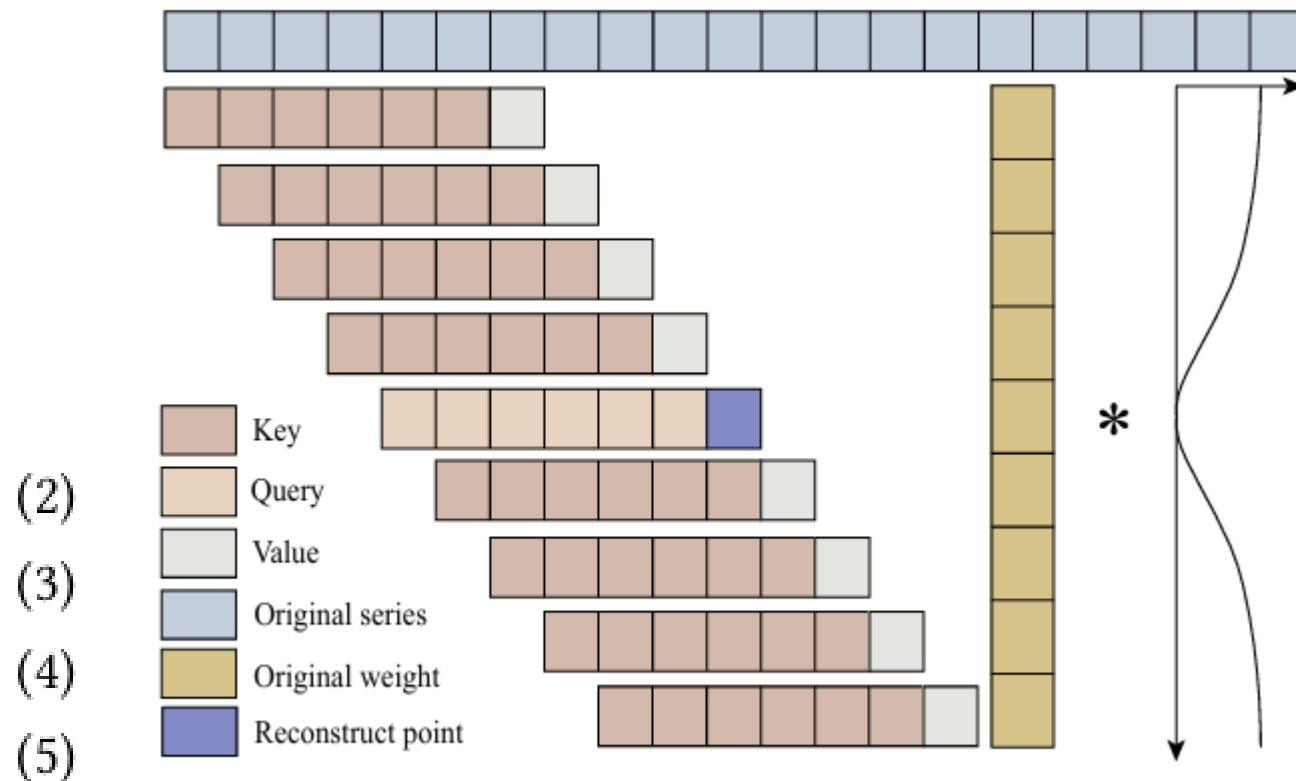
$$x \in R^{T \times D} \text{ to } \bar{x} \in R^{T \times (p+1) \times D}$$

$$\bar{x} = \text{SlidingWindow}(x, p + 1, 1),$$

$$q_d = \bar{x}[:, : p + 1, d],$$

$$k_d = \bar{x}[:, : p + 1, d],$$

$$v_d = \bar{x}[:, p + 1, d].$$



(b) A layer of Skimming Attention

3.2.1 Skimming Attention

- We apply a standard attention mechanism.
- We multiply a gate curve $G \in R^{T \times T}$ to the A.

$$G[i, j] = 1 - \exp^{-\frac{(i-j)^2}{\sigma^2}}$$

$$\mathcal{A} = q_d k_d^T,$$

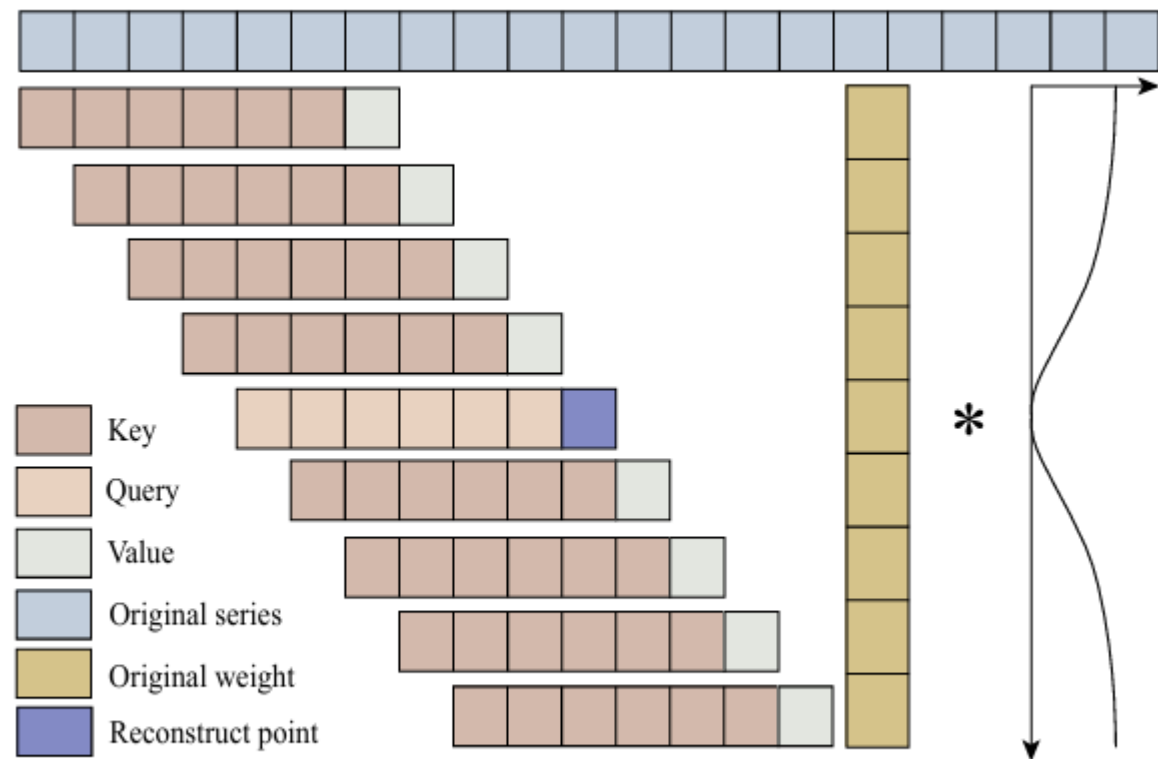
$$\tilde{\mathcal{A}} = \text{softmax}(\mathcal{A} * G),$$

$$\tilde{x}_l[:, d] = \tilde{\mathcal{A}} v_d.$$

(6)

(7)

(8)



(b) A layer of Skimming Attention

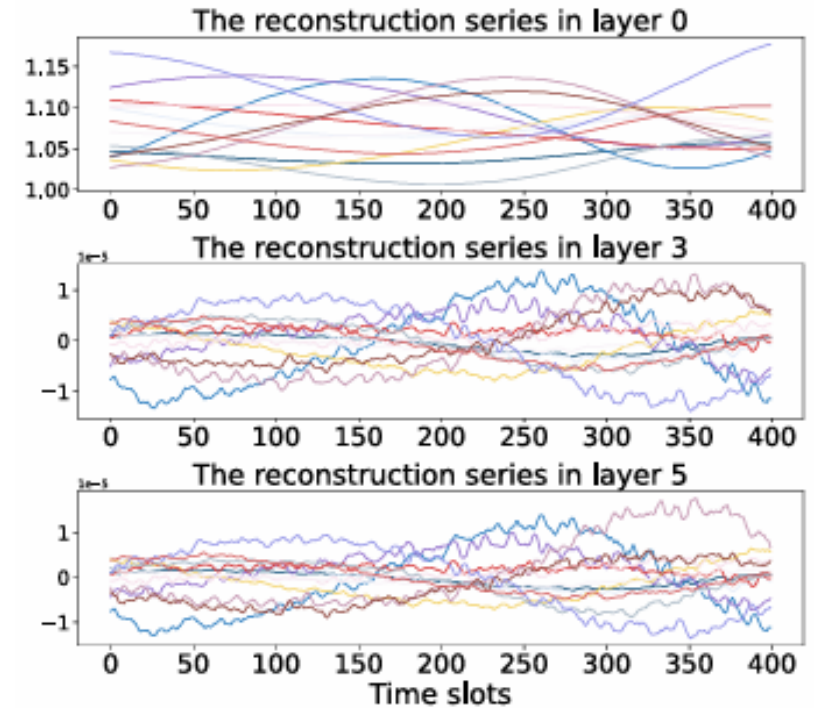
3.2.1 multi-layer Skimming Attention

- We organize different layers of skimming attention as follows to deal with compound periodic information.

$$\tilde{x}_l = \text{Skimming AttentionLayer}(x_l),$$

$$x_{l+1} = x_l - \tilde{x}_l,$$

- Where x^l is the l^{th} layer input data and $x_0 = x$.



(c) The series from different Skimming Attention layers

3.2.2 Neural OT

- Since we only pay attention to slow-down anomalies but not others (e.g., the execution speed of homework has significantly increased), we target modeling these non-periodic fluctuations but only hinder the reconstruction of slow-down anomalies. Inspired by the Optimal Transport (OT) algorithm, we transform a standard OT problem into a neural network and embed it into our model.

3.2.2 Neural OT

- Source distribution : reconstruction duration time distribution $\tilde{x}[t] \in R^{1 \times d}$.
- Target distribution : original duration time distribution $x[t] \in R^{1 \times d}$.
- Cost : $\langle P * \tilde{x}[t], C \rangle$.
- $P[d,s]$ denotes the ratio of $\tilde{x}[t,s]$ transporting to $\tilde{x}[t,d]$, $C[d,s]$ denotes the cost of transporting a unit from $\tilde{x}[t,s]$ to $\tilde{x}[t,d]$ and $*$ denotes element-wise multiplication.

$$C_{i,j} = \begin{cases} M[i] - M[j], & i > j \\ 0, & \text{else,} \end{cases} \quad (10)$$

- $M[i]$ is the midpoint of i^{th} interval in I (I is defined in Definition 3)

3.2.2 Neural OT

- Distribution after applying the transport strategy P to the source distribution : $P\tilde{x}[t]$.
- Optimization goal: $|P\tilde{x}[t] - x[t]|$.
- Constraint : $P^T \vec{1} = \vec{1}$
- λ is a hyperparameter belonging to $[0,1]$.

$$\begin{aligned} \min . \lambda < P * \tilde{x}[t], C > + |P\tilde{x}[t] - x[t]|_2, \\ \text{s.t.} P^T \vec{1} = \vec{1}, \end{aligned} \tag{11}$$

- Trainable parameter : P .
- Constraint : we manipulate P as $\text{softmax}(P^T)^T$, and the neural layer is specified as:

$$\hat{\tilde{x}} = \text{softmax}(P^T)^T \tilde{x} \tag{12}$$

3.2.3 Picky Loss Function

- We propose a picky loss function, which adaptively attributes a weight $W \in R^T$ according to trustworthiness to the loss of each time slot.

$$\mathcal{W} = \text{softmax}[(\mathcal{A} * \hat{G})\vec{1}]. \quad (13)$$

- We obtain the final loss function by attributing the weight W to each time slot and fusing the optimizing objective in Section 3.2, resulting in the final loss function as follows:

$$\mathcal{L} = \sum_{t=0}^T \mathcal{W}[t] (|\dot{\tilde{x}}[t] - x[t]|_2 + \lambda \langle P * \tilde{x}[t], C \rangle). \quad (14)$$

3.2.4 Anomaly Score

- We split the distribution intervals I according to the distribution density of task duration time. We use the difference between the task duration time expectations of the original distribution and reconstruction one as the anomaly score:

$$\begin{aligned}\text{AnomalyScore}[t] &= \mathbb{E}(\bar{T}(x[t])) - \mathbb{E}(\bar{T}(\hat{x}[t])) \\ &= \sum_{d=0}^D x[t, d] * M[d] - \sum_{d=0}^D \hat{x}[t, d] * M[d],\end{aligned}\tag{15}$$

- Anomaly Score[t] : anomaly score of t^{th} time slot.

4.1 Experiment Setup

- **Baseline Methods:** DCdetector, TranAD, AnomalyTransformer, VQRAE, OmniAnomaly, MSCRED, IASO, feature-shift detection.
- **Datasets :** We uses four datasets, two of which are industrial cloud cluster monitoring data(Ali1,Ali2), one is a public dataset(Mustang), and one is a synthetic dataset(Sync).

Table 1: Statistics of different datasets.

	Ali1	Ali2	Mustang	Sync
Dimension	14	14	17	14
Anomaly ratio (%)	3.71	6.06	7.75	1
Subsets	25	25	1	10

4.1 Experiment Setup

- **Hyperparameters** : We show some important hyperparameters in Table2, where we use Mut to stand for Mustang.
- **Evaluation Metrics** : We choose three of the most widely-used metrics to measure the performance of our method : the precision, recall, and F1 score.

Table 2: The hyperparameters.

Hyperparameter	Value	Hyperparameter	Value
Batch Size	100	Learning Rate	0.001
Skimming Layer of Ali1	10	Patch Size of Ali1	2
Skimming Layer of Ali2	6	Patch Size of Ali2	2
Skimming Layer of Mut	6	Patch Size of Mut	2
Skimming Layer of Sync	6	Patch Size of Sync	10

4.2 Prediction Accuracy

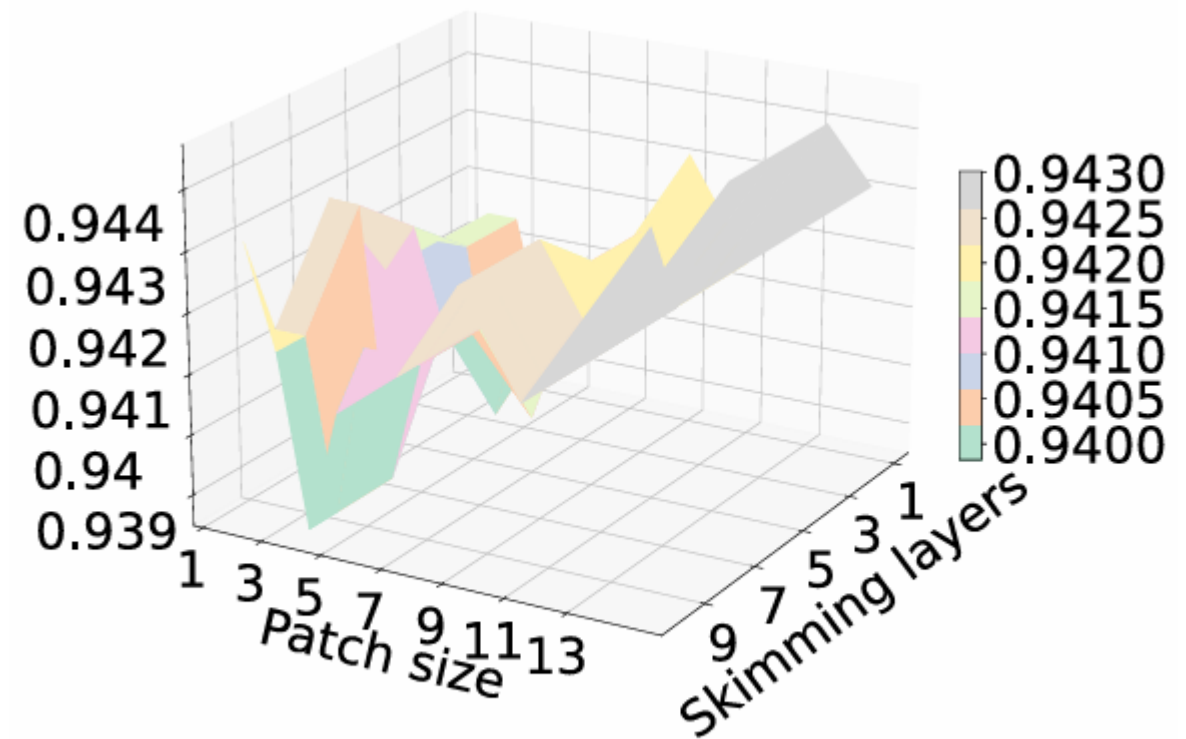
- We take 70% of each subset as the training set and take the remaining 30% as the testing set. For each subset, we train a unique model.

Table 3: Average performance of SORN and baselines on subsets of four datasets.

	Ali1			Ali2			Mustang			Sync		
	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1
MSCRED	0.841	0.981	0.878	0.928	0.988	0.954	0.871	0.960	0.896	0.717	0.874	0.779
Omni	0.681	0.981	0.782	0.814	0.987	0.890	0.812	0.968	0.878	0.655	0.997	0.787
AnomalyTr	1.000	0.870	<u>0.923</u>	0.999	0.763	0.857	1.000	0.891	<u>0.935</u>	1.000	0.680	<u>0.809</u>
TranAD	0.784	<u>0.989</u>	0.864	0.827	0.968	0.877	0.865	0.918	0.867	0.247	0.568	0.313
DCdetector	<u>0.984</u>	0.728	0.806	<u>0.994</u>	0.723	0.818	<u>0.968</u>	0.718	0.799	0.936	0.406	0.567
VRGAE	0.811	0.981	0.853	0.966	<u>0.992</u>	<u>0.978</u>	0.871	0.959	0.905	0.624	0.794	0.648
IASO	0.492	0.943	0.618	0.611	0.907	0.708	0.420	0.899	0.524	0.389	0.910	0.533
feature-shift	0.533	1.000	0.647	0.744	0.953	0.790	0.511	1.000	0.629	0.594	0.081	0.142
SORN [†]	0.891	<u>0.989</u>	0.897	0.955	0.997	0.968	0.895	0.996	0.916	<u>0.963</u>	0.893	0.919
SORN [‡]	0.944	0.969	0.939	0.960	0.967	0.955	0.912	0.971	0.919	0.939	0.832	0.874
SORN [§]	0.878	1.000	0.891	0.950	0.997	0.965	0.925	<u>0.996</u>	0.938	0.935	0.763	0.826
SORN	1.000	0.966	0.979	0.980	1.000	0.989	0.952	0.974	0.958	0.956	<u>0.926</u>	0.932

4.3 Hyperparameter Sensitivity

- We test the performance of SORN when setting the number of skimming layers and patch size as the Cartesian product of $\{1,3,5,7,9\}$ for skimming layers and $\{1,3,5,7,9,11,13\}$ for patch size.



(c) Parameter sensitivity

4.4 Ablation Study

- To evaluate the contribution of each module in SORN, we alternatively remove each submodule and test the performance of the remaining model.

Table 3: Average performance of SORN and baselines on subsets of four datasets.

	Ali1			Ali2			Mustang			Sync		
	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1	Pre	Rec	F1
MSCRED	0.841	0.981	0.878	0.928	0.988	0.954	0.871	0.960	0.896	0.717	0.874	0.779
Omni	0.681	0.981	0.782	0.814	0.987	0.890	0.812	0.968	0.878	0.655	0.997	0.787
AnomalyTr	1.000	0.870	<u>0.923</u>	0.999	0.763	0.857	1.000	0.891	<u>0.935</u>	1.000	0.680	<u>0.809</u>
TranAD	0.784	<u>0.989</u>	0.864	0.827	0.968	0.877	0.865	0.918	0.867	0.247	0.568	0.313
DCdetector	<u>0.984</u>	0.728	0.806	<u>0.994</u>	0.723	0.818	<u>0.968</u>	0.718	0.799	0.936	0.406	0.567
VRGAE	0.811	0.981	0.853	0.966	<u>0.992</u>	<u>0.978</u>	0.871	0.959	0.905	0.624	0.794	0.648
IASO	0.492	0.943	0.618	0.611	0.907	0.708	0.420	0.899	0.524	0.389	0.910	0.533
feature-shift	0.533	1.000	0.647	0.744	0.953	0.790	0.511	1.000	0.629	0.594	0.081	0.142
SORN [†]	0.891	<u>0.989</u>	0.897	0.955	0.997	0.968	0.895	0.996	0.916	<u>0.963</u>	0.893	0.919
SORN [‡]	0.944	0.969	0.939	0.960	0.967	0.955	0.912	0.971	0.919	0.939	0.832	0.874
SORN [§]	0.878	1.000	0.891	0.950	0.997	0.965	0.925	<u>0.996</u>	0.938	0.935	0.763	0.826
SORN	1.000	0.966	0.979	0.980	1.000	0.989	0.952	0.974	0.958	0.956	<u>0.926</u>	0.932

5. Conclusion

- We introduce SORN as a method for detecting cluster wide task slowdowns in cloud clusters , offering three distinctive features.
- Propose a method(Skimming Attention) to separately reconstruct sub-periodic components to ensure accurate reconstruction of both high and low amplitude subperiods.
- Neural OT , which selectively reconstructs non-slowng exceptional fluctuations.
- Picky Loss , which assigns weights to time slots in the loss function based on their reliability.
- Extensive experiments demonstrate that SORN outperforms state-of-the-art methods in real-world datasets.